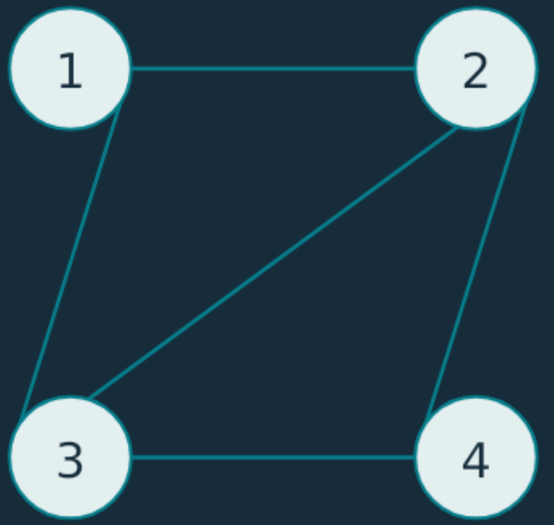
Graph neural networks

A tutorial in relationships, message passing, and working code

Nodes, edges, and feature vectors
Neighborhood aggregation and graph pooling
Training, evaluation, and prediction



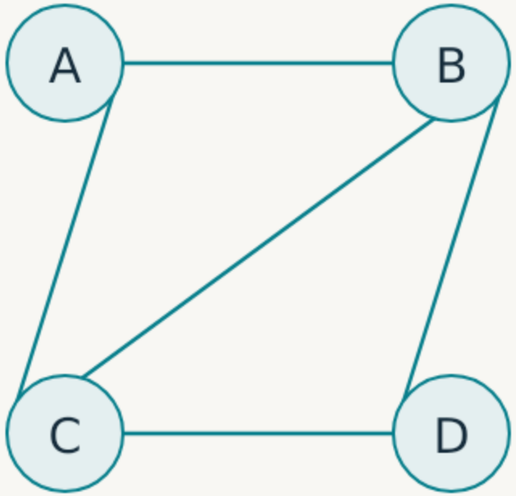
Learned neighbor updates

20 slides with detailed notes
Python + NumPy laboratory included

A graph needs structure and features

The adjacency matrix stores relationships. The feature matrix stores attributes.

Graph structure



Adjacency A

0	1	1	0
1	0	1	1
1	1	0	1
0	1	1	0

Rows / columns: A, B, C, D

Node features X

1	0.2
0	-0.6
1	0.7
0	0.1

Flag, measurement

A has shape [N, N]. X has shape [N, F]. Node ordering must agree.

Prediction can target nodes, edges, or graphs

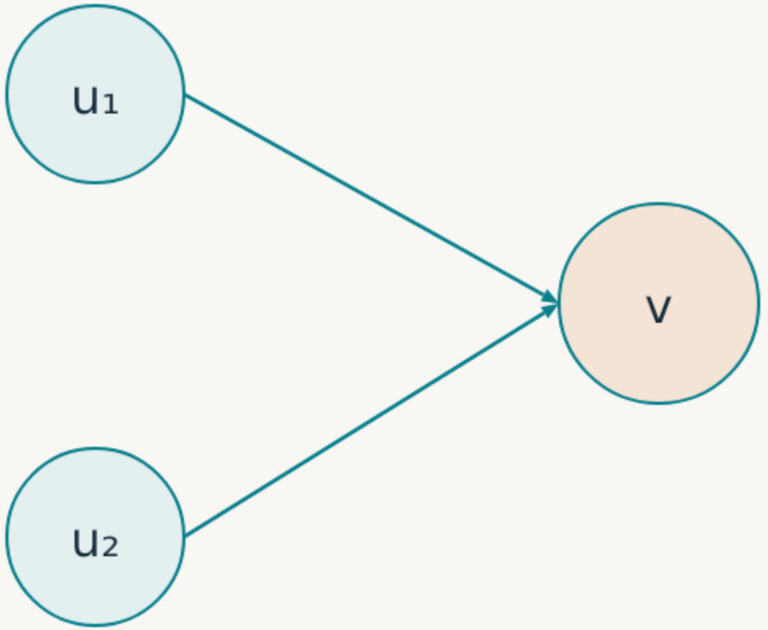
The location of the label determines the output head and the evaluation unit.

Task	Label placement	Example output
Node classification	One class for each labeled node	Node type or state
Edge / link prediction	One label for a candidate pair	Bond or interaction probability
Graph classification	One class per graph	Mostly same-type or different-type edges
Graph regression	One number per graph	Fraction of same-type edges

The included lab implements the two graph-level tasks.

Message passing updates each node

Neighbors provide messages; a shared update function combines them with the node’s current state.



$$m_v = \text{mean}\{h_u : u \text{ is a neighbor of } v\}$$

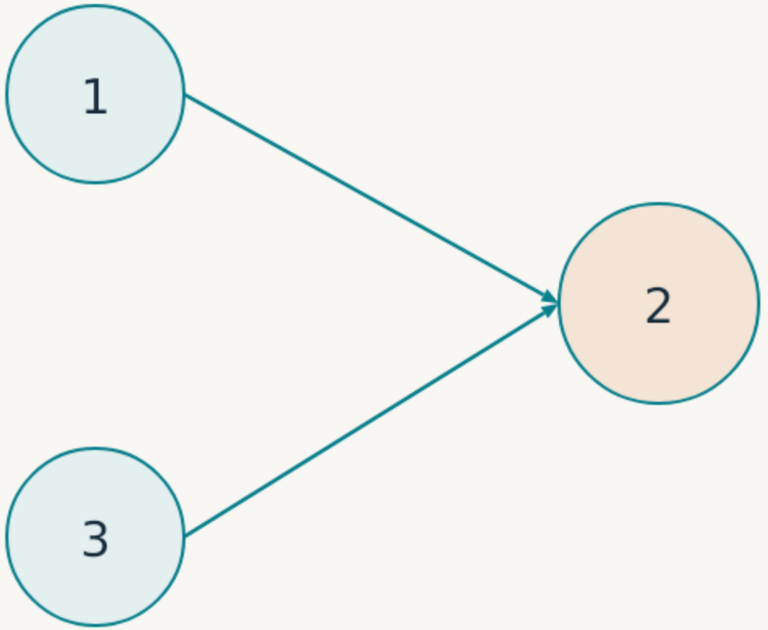
$$h_v' = \text{ReLU}(h_v W_s + m_v W_n + b)$$

Neighbor messages

The same Ws, Wn, and b apply to every node in the layer.

A neighbor update calculated by hand

A simple scalar example makes aggregation, weights, and bias concrete.



Shared scalar weights

Self weight Ws = 0.5
Neighbor weight Wn = 1.5
Bias b = −1

Updated center = 3

m = (1+3)/2 = 2 z = 2(0.5) + 2(1.5) − 1 = 3 h′ = 3

Mean aggregation as matrix multiplication

Row-normalized adjacency turns a sum of neighbors into their mean.

$d_v = \Sigma_u A[v,u]$ $\quad$ $M[v,u] = A[v,u] / d_v$

$$Z = H\,W_s + (M\,H)\,W_n + b$$

$$H' = \mathrm{ReLU}(Z)$$

Isolated node: its row of M is zero.

Self information has its own weights.

No self loops in the lab adjacency.

GCN, GraphSAGE, and graph attention

These families differ in how they transform and combine neighbor information.

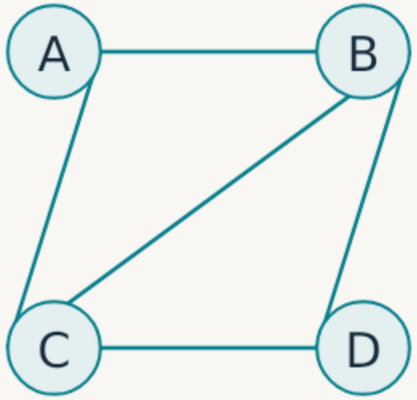
Family	Aggregation idea	Distinction
GCN	Self loops + symmetric degree normalization	One shared transformation in the basic layer
GraphSAGE-style mean	Mean of neighbors + separate self information	Inductive rule; sampling is optional for scale
Graph attention	Learned neighbor coefficients	Weights depend on node representations
This laboratory	Exact dense mean over all neighbors	Two layers, no sampling, no edge features

GCN: $H' = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H W)$, with $\tilde{A} = A + I$

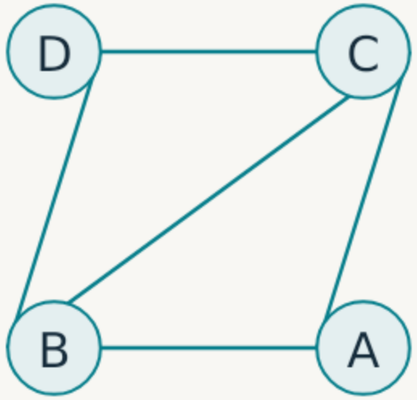
Node order must not change graph meaning

Node embeddings reorder with the nodes; a graph-level prediction stays the same.

One node ordering



A reordered drawing



$X' = PX$ $A' = PAP^T$

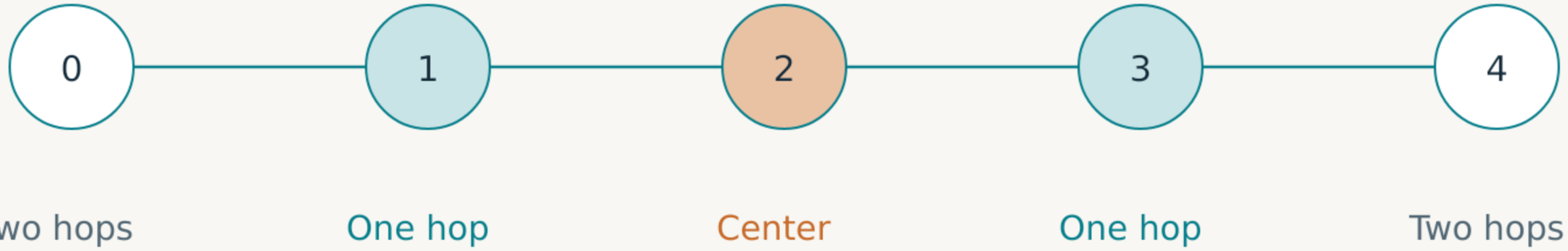
Node output: $H' = PH$

Graph output: $y' = y$

The unit test permutes features, both adjacency axes, and the validity mask together.

Depth controls the neighborhood reach

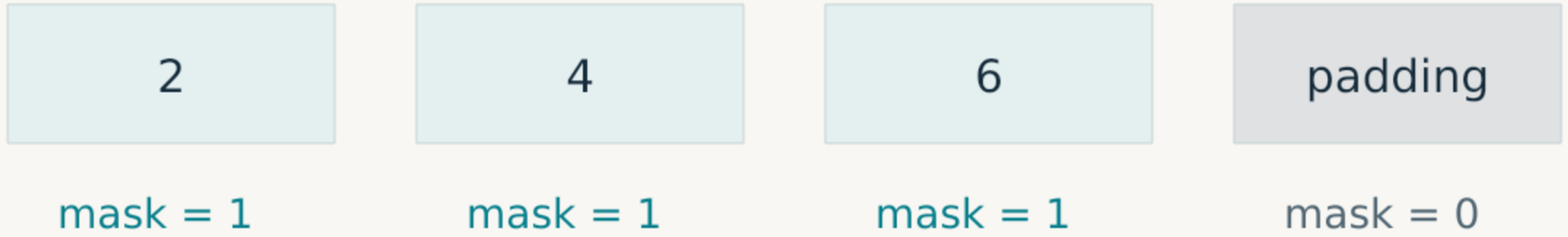
Two message-passing layers can use information from nodes up to two edges away.



Local information propagates one edge per layer. More layers also increase mixing and computation.

Graph pooling and validity masks

A graph readout summarizes node embeddings while ignoring padding.



Mean = (2 + 4 + 6) / 3 = 4

g = Σv maskv hv / Σv maskv

z = g Wo + bo

Use the true node count. Padding does not contribute to aggregation or readout.

Batching graphs

Small dense batches are easy to inspect; sparse disjoint batches are better for large graphs.

Quantity	Dense lab shape	Meaning
Node features X	B × Nmax × F	One feature row per node
Adjacency A	B × Nmax × Nmax	Independent graph structure per example
Validity mask	B × Nmax	1 for a real node, 0 for padding
Graph labels y	B	One target per graph

No cross-graph edges. The tutorial uses dense arrays for clarity, not for scale.

Node features, edge features, and labels

Encoding must preserve the scientific meaning of each attribute.

Attribute	Representation	Lab behavior
Logical node flag	0 or 1	Preserved as a Boolean-valued feature
Numerical node value	Finite floating point	Scaled using valid training nodes
Node category	One-hot or embedding	Discussed extension
Edge attribute	Message input or relation-specific map	Discussed extension
Graph label	Class ID or scalar	One target per whole graph

Reordering nodes is different from rotating coordinates. Each symmetry needs its own treatment.

Graph evaluation and leakage

A split defines which nodes, relationships, and graphs the model may see.

Inductive evaluation

Hold out complete graphs or future entities.

Split related specimens or structures together.

Fit transformations on training graphs.

Transductive and link tasks

State whether unlabeled structure is visible.

Remove held-out target edges when required.

Prevent future-edge and future-feature leakage.

Graph classification and regression

The same graph embedding can feed different prediction heads.

$$g = \text{masked_mean}(H_{final})$$

$$\text{Classification: } z = g\,W_o + b_o, \quad L = -\log p_{true}$$

$$\text{Regression: } \hat{y} = g\,w + b, \quad L = \text{mean}((\hat{y}-y)^2)$$

Target fraction = matching-flag edges / all edges

Class 1 if the fraction exceeds 0.5.

Labels are synthetic structural exercises.

Gradients flow through aggregation

The graph is fixed in this lab; feature and weight sensitivities follow ordinary matrix calculus.

$$\partial L/\partial W_s = H^T \Delta \qquad \partial L/\partial W_n = (MH)^T \Delta$$

$$\partial L/\partial H = \Delta\, W_s^T + M^T(\Delta\, W_n^T)$$

Pooling sends gradients only to valid nodes.

Sum parameter gradients over graphs and nodes.

Apply the ReLU derivative at each layer.

A and M remain fixed input structure.

Message passing in executable NumPy

Matrix multiplication makes the neighborhood computation visible.

```
M = normalize_rows(A)
H = X
valid = mask[..., None]
for Ws, Wn, b in layers:
    neighbor = M @ H
    Z = H @ Ws + neighbor @ Wn + b
    H = np.maximum(Z, 0) * valid

g = H.sum(1) / mask.sum(1, keepdims=True)
logits = g @ Wo + bo
```

2 input features
16 hidden features per layer
2 message-passing layers
Masked mean readout

642 parameters for 2 classes

Full runnable source and tests accompany the presentation.

A reproducible graph-learning harness

The checkpoint includes preprocessing, architecture, splits, and label conventions.

```
python run_all.py

python lab.py train --data data/classification.npz \
    --out runs/experiment_2 --lr 0.001
python lab.py evaluate --data data/classification.npz \
    --run runs/experiment_2
```

Batch training

Forward → loss → backward
AdamW update → validation

Saved checkpoint

Weights and feature statistics
Split indices and dataset hash

Independent testing

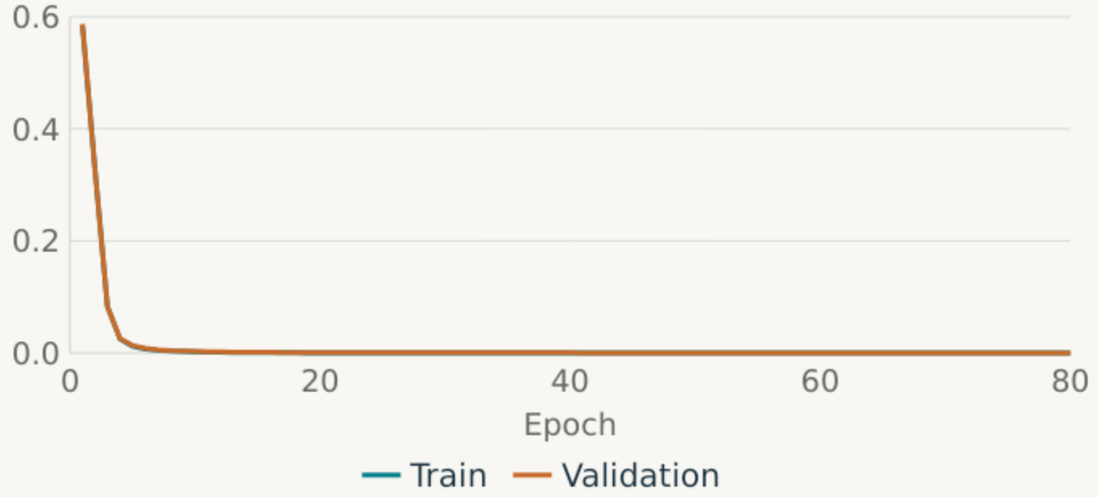
Restore selected weights.
Report held-out metrics.

Data archive: X, A, mask, y. Prediction archives omit y.

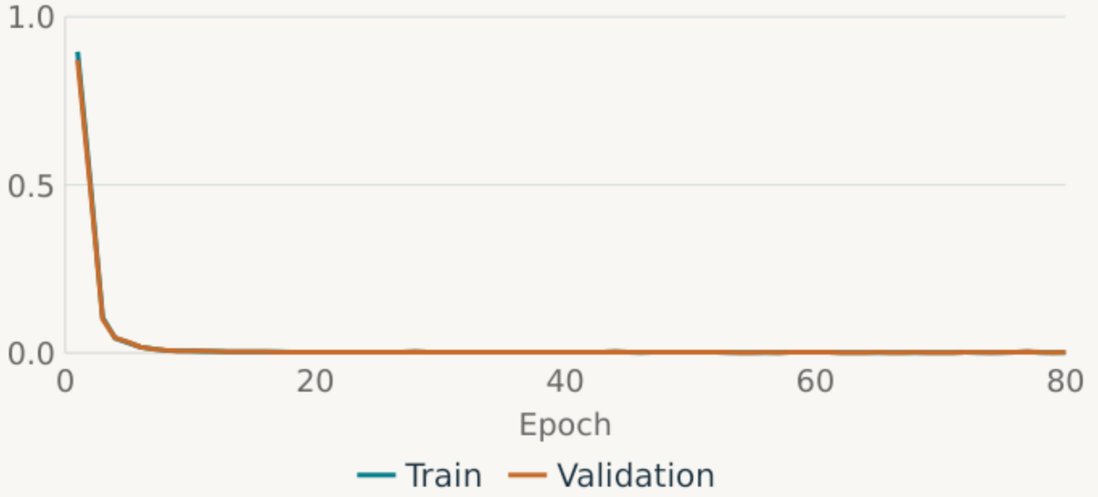
Measured results on held-out graphs

The model learns the synthetic relationship between node flags and graph edges.

Classification: cross-entropy



Regression: standardized MSE



Test accuracy 100.00%

Majority baseline: 51.42%
Macro-F1 1.0000
212 test examples; best epoch 80

Test RMSE 0.0111

Training-mean baseline: 0.2402
MAE 0.0081 R^2 0.9978
211 test examples; best epoch 79

Synthetic teaching tasks. These scores do not establish real-world performance.

Limits, diagnostics, and scaling

Message passing has structural limits as well as ordinary optimization and data problems.

Risk	Consequence	Useful response
Uninformative graph or attributes	Edges do not improve prediction	Compare feature-only and structural baselines
Too much repeated aggregation	Node representations become similar	Check depth, residuals, and normalization
Graph symmetries / limited readout	Different structures look identical	Examine features and aggregation expressivity
Dense N × N storage	Memory grows quadratically	Use sparse edge lists for larger graphs

Seven tests passed. Directed, typed, geometric, and temporal graphs need explicit extensions.

Laboratory and further reading

The source and notes connect each graph operation to its mathematical role.

Exercises

1. Verify the 642-parameter count.
2. Compare mean and sum readouts.
3. Add an edge attribute to the message.
4. Compare with feature-only and exact baselines.

Primary references

Gilmer et al., message passing
Kipf & Welling, GCN
Hamilton et al., GraphSAGE
Velickovic et al., GAT

Companion lab: source code, automated tests, datasets, reference checkpoints, training histories, test metrics, prediction examples, and complete lecture notes.